

Document number:	P0738R1
Date:	2018-11-14
Project:	C++ Programming Language, Library Working Group
Reply-to:	Casey Carter < Casey@Carter.net >

I Stream, You Stream, We All Stream for `istream_iterator`

Abstract

The specification and design of `istream_iterator` have some problems. First, the specification in the Standard begins with two paragraphs ([`istream.iterator`]/1 and /2) that intermix semi-normative description with actual normative requirements. This results in requirements that are either redundant, or are far from the entity whose behavior they are intended to describe. These normative requirements should be in the specification of the individual member functions. The current situation is both confusing and inconsistent with the specification of other library components.

Second, the semantics of exactly when an `istream_iterator` performs a read from its underlying input stream are unclear. The specification purports to allow an implementation to delay reading the initial value from the stream, which has been a source of confusion in the past ([LWG 245 “Which operations on `istream\_iterator` trigger input operations?”](#)). This program, for example:

```
istream_iterator<int>{cin};
istream_iterator<int>{cin};
istream_iterator<int>{cin};
istream_iterator<int>{cin};
istream_iterator<int>{cin};
```

is specified to read between zero and five integers from the standard input. We argue that an implementation that delays reading the initial value from the stream cannot, in fact, conform to the input iterator requirements.

While correcting these two specification problems we also propose some cleanup and modernization of the specification of `istream_iterator` in passing.

Revision History

Revision 1

- Struck proposed modifications to the Ranges TS, which is effectively dead after merging [P0896R4 “The One Ranges Proposal”](#) to the working draft.
- Rebase proposed changes onto the post-San Diego working draft.
- Updated the style of specification to use the specification elements introduced by [P0788R3 “Standard Library Specification in a Concepts and Contracts World”](#).
- Split “this thingy is a constexpr thingy” wording out of “Effects” and into “Remarks”.

Discussion

`istream_iterator`’s jumbled introduction

The presentation of many Standard Library classes follows a common structure:

- a paragraph with a brief general introduction
- a class synopsis
- a few paragraphs of class-specific normative requirements (“template parameter T must be an object type that meets the `Destructible` requirements” ... “template parameter T may be an incomplete type”)
- a detailed specification of the class’s member functions and their individual behaviors

`istream_iterator` does not follow that structure, despite appearing to do so. Its introductory paragraphs are *not* brief, and verge on tutorial: “It is impossible to store things into `istream` iterators.” It contains normative requirements that in some cases duplicate requirements in the specification of the individual member functions (“Two end-of-stream iterators are always equal”), and in other cases are the only occurrence of a requirement that *should* appear in the specification of a member function (“If the iterator fails to read and store a value of T (`fail()` on the stream returns `true`), the iterator becomes equal to the end-of-stream iterator value”). Removing duplicate requirements and relocating non-duplicate requirements to the specification of the entity to which they apply would improve the quality and consistency of the specification.

Confused postcondition

The specification of `istream_iterator`’s constructor from `istream_type&` ([`istream.iterator.cons`] para 3 and 4):

```
istream_iterator(istream_type& s);
```

3 *Effects*: Initializes `in_stream` with `std::addressof(s)`. `value` may be initialized during construction or the first time it is referenced.

4 *Postcondition*: `in_stream == &s`.

The postcondition in para 4 is (a) redundant with the effect “initializes `in_stream` with ...”, and (b) flat out wrong if the implementation tries to read the first value from the stream and immediately hits end-of-stream. We propose simply removing this postcondition paragraph.

Delayed-initialization semantics

`istream_iterator` purports to allow implementations that delay reading the first value from the stream until it is needed ([`istream.iterator.cons`]/3 “`value` may be initialized during construction or the first time it is referenced”). Consider this program fragment:

```
istream_iterator<int> i1(cin);
auto i2 = i1;
assert(*i1 == *i2);
```

We claim that this program does not assert. In the Ranges TS, iterator copies must be *equal* - meaning they can be substituted into expressions designated as equality-preserving - and `*i` is exactly such an expression. Since there are no intervening modifications between the copy construction of `i2` and the assertion, it must be the case that `*i1 == *i2`. For Standard C++, the semantics are less clear: copies are required to be equivalent (Table 24 `CopyConstructible` requirements), although the meaning of the term “equivalent” in this context is not clearly defined. It’s not unreasonable to interpret “equivalent” in this context to mean something similar to the more concrete semantics given in the Ranges TS. One of the primary goals of the Ranges TS is to more clearly specify the semantics of the standard library for cases such as this, and presumably the TS WP reflects WG21’s intent for iterators and algorithms fairly well.

An implementation of `istream_iterator` that reads the initial value on construction and never delays initialization obviously satisfies the preceding requirements: the value stored in `i1` is copied into `i2`, those copies are obviously equal in the assertion. Can an implementation that delays initialization meet that bar?

For an implementation that delays initialization to work it must read the initial value from the stream sometime between the construction of `i1` and the dereference of `i1` in the assertion. That leaves two possible points for the delayed init to occur:

- in the copy constructor that initializes `i2` from `i1`. This is not possible, since [\[istream.iterator.cons\]/5](#) requires the copy constructor to be trivial when `T` is a trivially copyable (Standard) / literal (Ranges TS) type.
- in the first evaluated dereference operator that reads the value of `i1` or `i2`. For this to work the *second* dereference operator evaluation must see the same value, so there must be some connection between the two iterator objects that is set up in the trivial copy constructor and torn down again in the trivial destructor. We do not believe that forming such an association between two objects is possible given the constraints that the copy constructor and destructor must be trivial: any external memory/resource used to coordinate communication between the objects must necessarily leak.

On the basis of this argument that a conforming implementation *cannot* delay initialization, we propose to remove the allowance to do so thereby simplifying the specification and clarifying the semantics of `istream_iterator`.

Technical Specifications

All wording relative to the post-San Diego C++ working draft.

Strike all but the first sentence of [\[istream.iterator\]/1](#), and the text of [paragraph 2](#):

1 The class template `istream_iterator` is an input iterator ([\[input.iterators\]](#)) that reads ~~(using `operator>>`)~~ successive elements from the input stream for which it was constructed. ~~After it is constructed, and every time `++` is used, the iterator reads and stores a value of `T`. If the iterator fails to read and store a value of `T` (`fail()` on the stream returns `true`), the iterator becomes equal to the end-of-stream iterator value. The constructor with no arguments `istream_iterator()` always constructs an end-of-stream input iterator object, which is the only legitimate iterator to be used for the end condition. The result of `operator*` on an end-of-stream iterator is not defined. For any other iterator value a `const T&` is returned. The result of `operator->` on an end-of-stream iterator is not defined. For any other iterator value a `const T*` is returned. The behavior of a program that applies `operator++()` to an end-of-stream iterator is undefined. It is impossible to store things into `istream` iterators. The type `T` shall satisfy the `Cpp17DefaultConstructible`, `Cpp17CopyConstructible`, and `Cpp17CopyAssignable` requirements.~~

2 ~~Two end-of-stream iterators are always equal. An end-of-stream iterator is not equal to a non-end-of-stream iterator. Two non-end-of-stream iterators are equal when they are constructed from the same stream.~~

Add a new paragraph to the end of [\[istream.iterator\]](#), after the class synopsis:

~~-?- The type `T` shall meet the `Cpp17DefaultConstructible`, `Cpp17CopyConstructible`, and `Cpp17CopyAssignable` requirements.~~

Modify [\[istream.iterator.cons\]](#) as follows:

```
constexpr istream_iterator();  
constexpr istream_iterator(default_sentinel_t);
```

1 *Effects*: Constructs the end-of-stream iterator, [value-initializing value](#). ~~If `is_trivially_default_constructible_v<T>` is true, then these constructors are `constexpr` constructors.~~

2 Ensures: `in_stream == nullptr`.

-?- Remarks: If the initializer `T(.)` in the declaration `auto x = T(.);` is a constant-initializer ([basic.start.static]), then these constructors are constexpr constructors.

```
istream_iterator(istream_type& s);
```

3 Effects: Initializes `in_stream` with `addressof(s)`, value-initializes `value`, and then calls `operator++()`. ~~value may be initialized during construction or the first time it is referenced.~~

4 ~~Ensures: `in_stream == addressof(s)`.~~

```
istream_iterator(const istream_iterator& x) = default;
```

5 Effects: Constructs a copy of `x`. ~~If `is_trivially_copy_constructible_v<T>` is true, then this constructor is a trivial copy constructor.~~

6 Ensures: `in_stream == x.in_stream`.

-?- Remarks: If `is_trivially_copy_constructible_v<T>` is true, then this constructor is trivial.

```
~istream_iterator() = default;
```

7 ~~Effects: The iterator is destroyed.~~ Remarks: If `is_trivially_destructible_v<T>` is true, then this destructor is trivial.

Modify [\[istream.iterator.ops\]](#) as follows:

```
const T& operator*() const;
```

-?- Expects: `in_stream != nullptr`.

1 Returns: `value`.

```
const T* operator->() const;
```

-?- Expects: `in_stream != nullptr`.

2 Returns: `addressof(operator*()value)`.

```
istream_iterator& operator++();
```

3 ~~Requires~~Expects: `in_stream != nullptr`.

4 Effects: ~~As if by:~~

~~`*in_stream >> value;`~~`if (!(*in_stream >> value)) in_stream = nullptr;`

5 Returns: `*this`.

```
istream_iterator operator++(int);
```

6 ~~Requires: `in_stream != 0`.~~

7 Effects: ~~As if by:~~Equivalent to:

```
istream_iterator tmp = *this;
```

~~`*in_stream >> value;`~~

```
++*this;  
return {tmp};
```

[...]

Acknowledgements

I would like to thank Tim Song for pointing out to me that `istream_iterator::operator*` requires the iterator to not be an end-of-stream iterator, and that this requirement is squirreled away in [\[istream.iterator\]/1](#) and NOT in [\[istream.iterator.ops\]](#) with the specification of `operator*` where a sane person would expect it to be.